WELL KILLER

LoRa Irrigation Well Control System

Final Software Development Project Report

C++ | Object-Oriented Design | Layered Architecture | UML | Git/GitHub

Final Submission - August 2026

1. Project Overview

Well Killer is a C++ application that simulates a LoRa-based irrigation well monitoring and emergency shutdown system. The application addresses a real agricultural safety problem: a farmer may need to quickly identify the operating state of irrigation wells and remotely stop one or all wells during a problem such as severe weather, equipment trouble, or another unsafe condition. The software models the home LoRa transmitter, the well-side LoRa receiver, well status, a Murphy shutdown circuit, and a persistent shutdown log.

The current software provides more than the required three core features. Its primary features are:

- Display the operating status of all configured irrigation wells.
- Shut down a selected well through a simulated LoRa command sequence.
- Perform an emergency shutdown of all running wells.
- Record shutdown events to a persistent text log and display the log.
- Test the simulated LoRa transmitter-to-receiver communication path.

2. Requirements and Scope

The final project was required to solve a real-world problem, contain at least three substantial features, use object-oriented design, follow a layered or modular architecture, include UML models, use multiple testing strategies, use Git for version control, and provide user/developer documentation. Well Killer was designed around those requirements while keeping the implementation understandable and maintainable.

2.1 Functional Requirements

ID	Requirement
FR-1	The system shall display each configured well and whether it is RUNNING or STOPPED.
FR-2	The user shall be able to select one well and request a remote shutdown.
FR-3	The system shall simulate transmission and receipt of a LoRa shutdown command.
FR-4	The user shall be able to initiate an emergency shutdown of every running well after confirmation.
FR-5	The system shall record successful shutdown events in a text log.
FR-6	The user shall be able to display the shutdown log.
FR-7	The user shall be able to test simulated LoRa communication.
FR-8	The system shall reject invalid menu choices and invalid well numbers without terminating unexpectedly.

2.2 Non-Functional Requirements

- Usability: clear opening instructions, a numbered menu, warnings, confirmations, and status messages keep the user informed.
- Reliability: invalid input is handled with exceptions and shutdown commands are validated before changing well state.
- Maintainability: classes separate device behavior, well state, and application control logic; source files are being separated as the project is refactored.
- Traceability: shutdown events are written to a persistent log and source changes are tracked in Git/GitHub.
- Safety: the emergency all-well stop requires explicit confirmation before execution.

3. Object-Oriented Design and Architecture

The application uses object-oriented design to represent both physical devices and irrigation wells. The design avoids putting all behavior in main(). Instead, responsibilities are divided among several classes and the user-interface functions.

Class	Primary Responsibility
Device	Abstract base class containing the device name and pure virtual communicate() behavior.
LoRaTransmitter	Derived Device that simulates transmitting test signals and well

	shutdown commands.
LoRaReceiver	Derived Device that simulates listening for test signals and receiving shutdown commands.
Well	Encapsulates well ID, well name, and running state; exposes controlled operations such as shutdown() and displayStatus().
WellController	Coordinates wells, LoRa devices, logging, single-well shutdown, all-well shutdown, status display, and communication tests.

3.1 OOP Principles Used

Encapsulation: Well keeps its ID, name, and running state private. State is accessed or changed through public member functions.

Inheritance: LoRaTransmitter and LoRaReceiver derive from the Device base class.

Polymorphism: Device defines virtual communicate() behavior. The transmitter and receiver override it, and the controller can call the appropriate implementation through Device pointers.

Abstraction: The Device base class describes the common concept of a communicating device without requiring the caller to know implementation details.

Overloading: The controller supports well shutdown by integer ID and by string name, giving two interfaces for the same operation.

3.2 Layered Modular Architecture

For this project, a lightweight layered modular architecture was selected instead of a heavyweight MVC or microservice design. The layers are:

1. Presentation Layer - opening screen, menu, user input, confirmations, and console output.
2. Application/Control Layer - WellController coordinates use cases such as status display, individual shutdown, emergency shutdown, diagnostics, and log viewing.
3. Domain/Device Layer - Well, Device, LoRaTransmitter, and LoRaReceiver represent the system objects and behaviors.
4. Persistence/Hardware Boundary - text-file logging and the simulated LoRa/Murphy hardware interaction.

4. Development Process: Agile and Unified Process

The project can be described as an iterative Agile development effort carried out within the four major phases of the Unified Process (UP). This combination fits the project because small features were added and tested incrementally while the overall work still progressed from concept to architecture, implementation, and final delivery.

UP Phase	Well Killer Activities
Inception	Defined the agricultural safety problem, target user (farmer/operator), core use cases, project scope, and minimum features.
Elaboration	Refined requirements, identified classes and responsibilities, selected a layered modular architecture, and produced UML models.
Construction	Implemented the C++ console application, LoRa simulation, shutdown logic, exception handling, file logging, and repository organization.
Transition	Rebuilt and ran the application, prepared GitHub for submission, added UML documentation, and produced final user/developer documentation.

4.1 Agile Iterations

- Iteration 1 - establish Well objects, operating status, and menu-driven UX.
- Iteration 2 - add single-well shutdown and validation.
- Iteration 3 - add transmitter/receiver classes and LoRa communication simulation.
- Iteration 4 - add emergency all-well shutdown and persistent logging.
- Iteration 5 - improve exception handling, UX, documentation, and source organization.
- Iteration 6 - add UML, GitHub documentation, testing documentation, and final submission materials.

5. UML Modeling

Four UML views were prepared to describe the system from different perspectives. The diagrams are also stored separately with the project documentation.

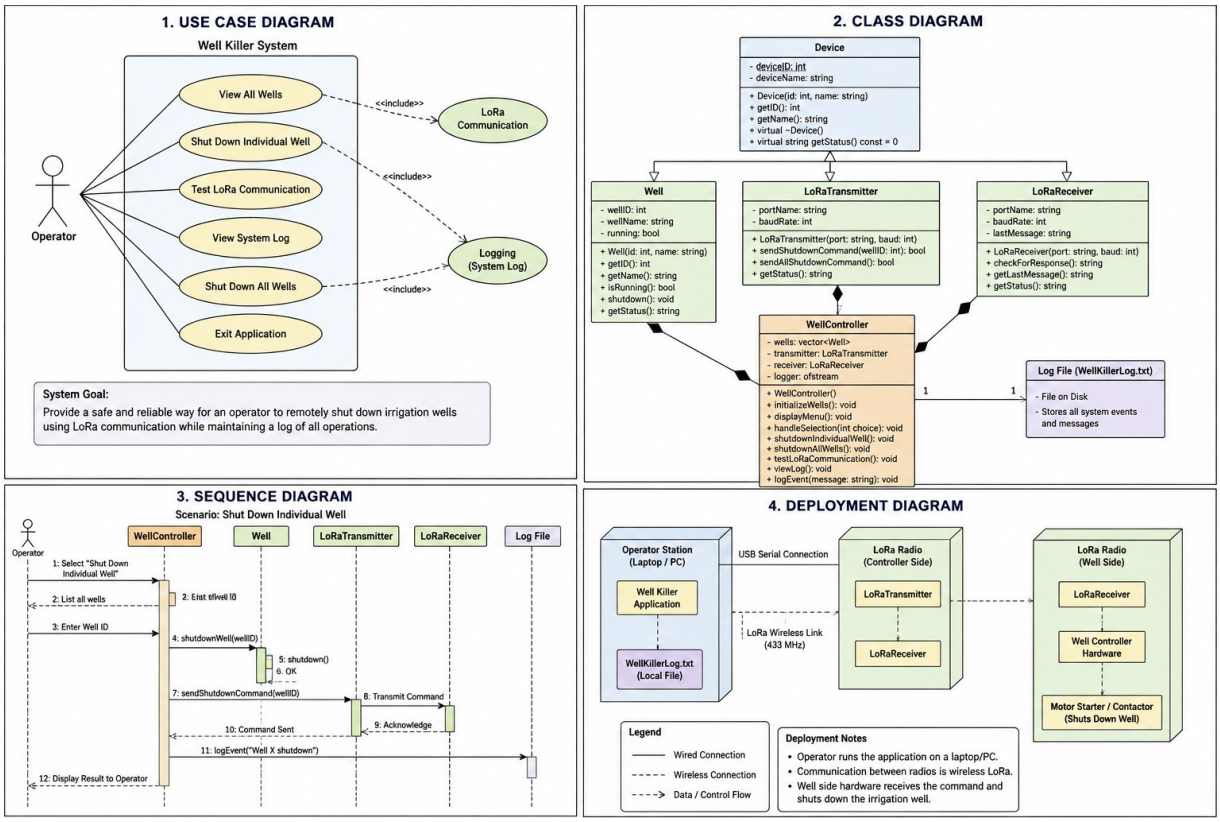


Figure 1. Well Killer UML overview: use case, class, sequence, and deployment views.

Use Case Diagram: Shows the farmer/operator interacting with the system to view status, shut down one well, stop all wells, view the log, and test LoRa communication.

Class Diagram: Shows Device inheritance, transmitter and receiver specialization, Well encapsulation, and WellController coordination.

Sequence Diagram: Models the single-well shutdown scenario: the operator selects a well, the controller validates it, the transmitter sends the command, the receiver receives it, the Murphy shutdown is triggered, the Well state changes, and the event is logged.

Deployment Diagram: Represents the expected physical topology: user/control computer, home LoRa transmitter, wireless LoRa link, well-side receiver/controller, Murphy shutdown circuit, and irrigation well/engine.

6. Testing Strategy

The project requirement calls for unit, integration, and system testing. The final submission documents all three levels. The current application was rebuilt and executed successfully after the source refactor. The following cases define the intended repeatable test coverage and the manual system checks used during development.

6.1 Unit Test Cases

ID	Test	Input/Action	Expected Result
UT-1	Construct Well	Create Well(1, "North Well")	isRunning() returns true.
UT-2	Shutdown Well	Call shutdown() on a running Well	isRunning() returns false.
UT-3	Invalid low ID	Call shutdownWell(0)	out_of_range exception is handled.
UT-4	Invalid high ID	Call shutdownWell(8) with seven wells configured	out_of_range exception is handled.
UT-5	Already stopped well	Shutdown the same well	Second request reports that

		twice	the well is already stopped and does not repeat the state change.

6.2 Integration Test Cases

ID	Integration	Action	Expected Result
IT-1	Controller + Well	Request Well 3 shutdown through WellController.	Well 3 changes from RUNNING to STOPPED.
IT-2	Controller + LoRa devices	Run a single-well shutdown.	Transmitter and receiver messages appear in the expected order.
IT-3	Controller + log file	Shut down a running well and then view the log.	A new shutdown record for that well is displayed.
IT-4	Emergency stop + wells	Confirm all-well emergency stop.	Every previously running well is STOPPED.

6.3 System Test Scenarios

ID	Scenario	Procedure	Expected Result
ST-1	Normal shutdown workflow	Launch app -> display status -> choose one well -> shut down -> redisplay status.	Selected well shows STOPPED; application remains operational.
ST-2	Emergency shutdown workflow	Launch app -> choose all-well stop -> confirm Y.	All running wells shut down and user receives completion message.
ST-3	Input validation	Enter non-numeric menu input or an out-of-range option.	Error message is shown and program returns to the menu.
ST-4	LoRa diagnostic	Choose Test LoRa Communication.	Transmitter and receiver test messages complete and test reports PASSED.
ST-5	Persistence check	Shut down a well -> exit -> rerun -> view log.	Previously written shutdown record remains in the text log.

Note: the present submission documents the full three-level testing strategy and successful manual build/run validation. Automated unit tests can be expanded in a future iteration without changing the system architecture.

7. Version Control and Repository Management

Git and GitHub were used to maintain the codebase. The project was published to a GitHub repository, a README was added, Visual Studio-generated files were excluded with .gitignore, and development work was performed on a refactor-layered-architecture branch before being merged into main. UML documentation was also added to the repository.

- main - submission-ready stable branch.
- refactor-layered-architecture - feature branch used to reorganize the Well class and project structure.
- Meaningful commits were created for the initial application, README/documentation, source refactor, and UML artifacts.
- .gitignore prevents local Visual Studio temporary/build files from polluting version control.

8. Deployment and Operation

The current deliverable is a desktop C++ simulation built with Microsoft Visual Studio. In deployment, the software represents a control station that communicates with a home LoRa transmitter. A wireless LoRa link carries a shutdown command to the receiver at the irrigation well. The receiver would interface with a Murphy shutdown circuit that stops the engine or irrigation equipment. The simulation deliberately separates software behavior from the final physical hardware so the control workflow can be tested safely before full field deployment.

9. Limitations and Future Improvements

- Replace simulated LoRa messages with live Arduino/LoRa serial communication.
- Read real engine or well state instead of assuming the seven simulated wells start in RUNNING state.
- Add acknowledgements, timeout/retry behavior, and failed-communication handling.
- Expand automated Google Test coverage for Well and WellController.
- Move remaining classes into separate header/source files to make the layered architecture even clearer.
- Add timestamps and richer event data to the shutdown log.
- Add authentication or command authorization before deploying remote shutdown capability in the field.

10. Developer Notes

The project should be opened and built in Microsoft Visual Studio as a C++ project. The repository contains the C++ source, Visual Studio project configuration, README documentation, a .gitignore file, and UML documentation. When contributing changes, create a focused branch, build and run the application, commit with a descriptive message, push the branch, and merge into main only after the change is verified.

11. Conclusion

Well Killer satisfies the central goal of the final project by applying software engineering practices to a real-world irrigation safety problem. The application contains multiple meaningful features, demonstrates core object-oriented principles, uses a layered modular design, includes UML models, defines unit/integration/system testing strategies, and is maintained in Git/GitHub. The project also provides a realistic path from software simulation to a future LoRa-connected hardware system controlling Murphy irrigation shutdown equipment.